A Minimal X.600 MCU Gateway for High-Density Video Conferencing on Alpine Linux in Kubernetes

For Zen Crypted X.422 Budda Protocol

Namdak Tonpa, CTO of Zen Crypted

July 12, 2026

Contents

Abstract

This paper presents the architectural design, implementation, and systems capacity review of `rtp`, a Multipoint Control Unit (MCU) gateway designed for high-density, real-time video conferencing in isolated distributed network namespaces. Traditional architectures (e.g., Selective Forwarding Units and headless-browser-based recording egress systems) suffer from high CPU and memory overhead. We propose a collapsed, monolithic Erlang/GStreamer node deployed as share-nothing Alpine Linux pods. By binding all participants of a room to the same pod instance using Ingress Sticky Sessions and disabling Erlang cluster distribution, the system scales horizontally without inter-node cluster link storms. Real-time video grid compositing (2x2 layout in 1080p) and audio mixing are delegated directly to a local, lightweight C99 GStreamer port binary, exchanging WebRTC SDP/ICE signaling lines directly over standard input and output pipes. We show that this model supports up to 10,000 concurrent signaling connections and reduces media encoding resource requirements by over 60% when hardware-accelerated.

1 Introduction

Real-time audio and video conferencing in distributed networks requires both high reliability (liveness) and low computational overhead (high room density per physical host). Typical WebRTC architectures utilize a Selective Forwarding Unit (SFU) where each participant uploads one stream and receives $K - 1$ downstream feeds from other participants. At scale, this places heavy download bandwidth and decoding overhead on client browsers, especially for low-powered endpoint terminals.

To address this, we implement a Multipoint Control Unit (MCU) model, where the server decodes all incoming streams, mixes the audio signals, composites the video tracks into a unified grid matrix, and broadcasts a single combined stream back to the clients. This reduces client-side complexity to $O(1)$.

To minimize server-side overhead (which traditionally relies on heavy headless browsers like Chromium to capture grid layouts), we present a C99 GStreamer-based MCU integrated directly into a share-nothing Erlang/OTP monolith pod running on Alpine Linux.

2 System Architecture

The system consists of three distinct layers: the proxy/routing ingress, the Erlang orchestrator, and the GStreamer media compositor.

2.1 Architectural Topology

Figure 1 shows the physical and logical entities, protocols, and network connections.

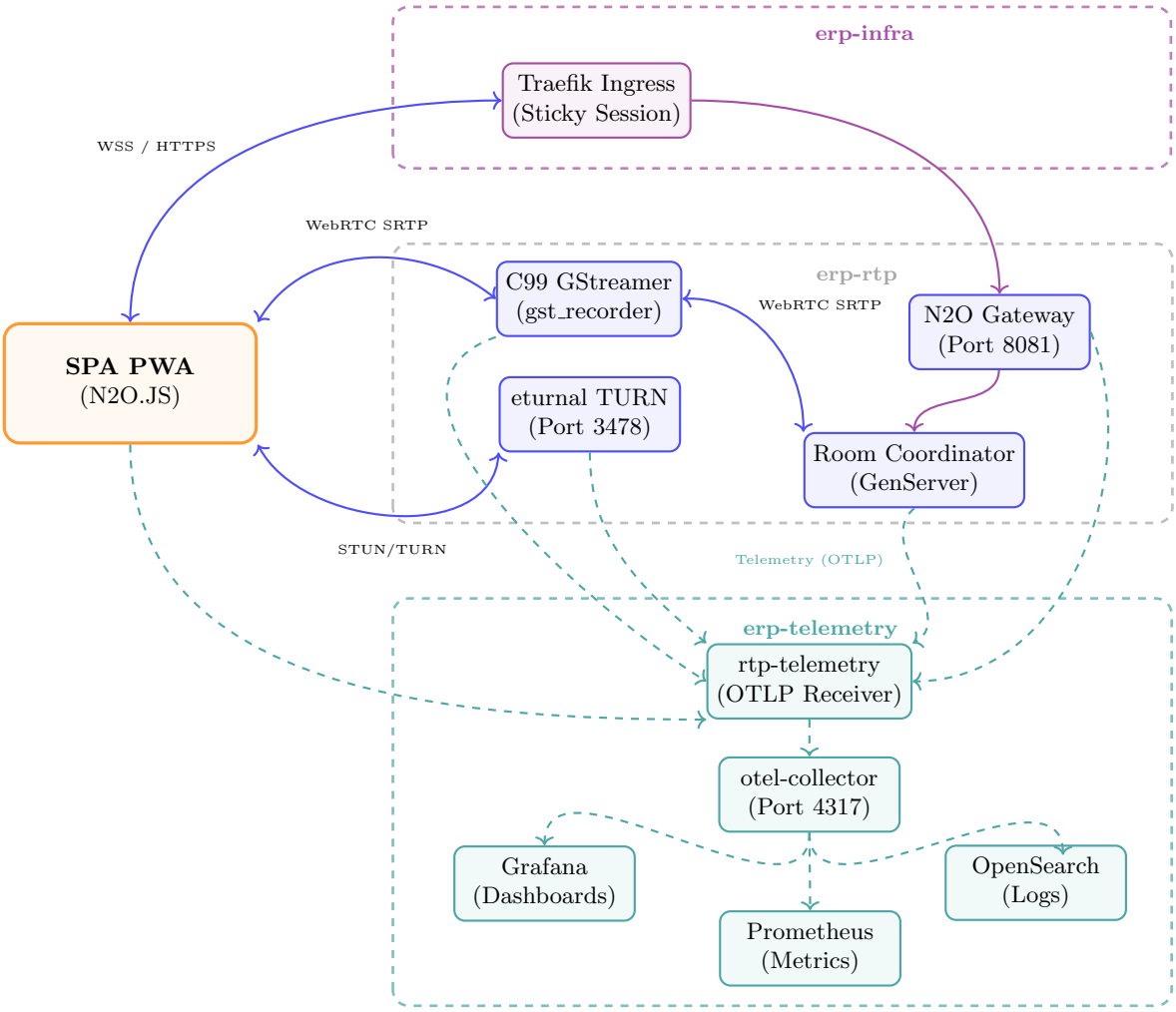

Figure 1: Three-namespaces architecture.

2.2 Ingress Sticky Routing & Room Sharding

To scale the horizontal architecture to 1,000+ concurrent rooms, we bypass Erlang distribution. The Ingress proxy (Nginx or Envoy) parses the Room ID from the connection URI path (e.g., `/room/{room_id}/signaling`) and maps it to a consistent target pod IP using a hash ring:

$$\text{Pod IP} = \text{Hash}(\text{room\_id}) \pmod{P} \tag{1}$$

Where P is the total count of running replicas. This ensures that:

- All participants of a specific room land on the same pod.
- The room signaling pub/sub is fully handled in-memory locally using N2O.
- The Erlang nodes run with no distribution, avoiding the $O(P^2)$ storm.
- Mnesia acts as a purely local persistent transaction ledger mounted on an Alpine Linux local directory PVC.

2.3 WebRTC Signaling Loop & C99 Port Protocol

The C99 GStreamer binary (`gst_recorder`) is spawned as an OS process by the Erlang supervisor. Rather than linking against heavy socket libraries, it reads signaling data from `stdin` and outputs event payloads to `stdout`.

2.3.1 Protocol Stack and Specifications

The monolith interfaces utilize a diverse set of network and inter-process communication (IPC) protocols to coordinate signaling, manage media routing, and report client telemetry. Table 1 provides a comprehensive description of each protocol, its transport layer, and operational parameters.

2.3.2 JSON Replacement Strategy on Existing Connections

The current implementation uses JSON for several high-frequency control and signaling paths. We propose replacing these with structured binary formats based on relevant ITU-T X-series Recommendations (from `synrc.com/standards.txt`), particularly the multi-peer, group, and managed P2P frameworks.

Selected ITU-T X-Stack Standards The following recommendations from the X-series provide a formal framework for group video conferencing:

- **X.601** — Multi-peer communications framework
- **X.602** — Group management protocol
- **X.603 / X.603.1 / X.603.2** — Relayed multicast protocol (simplex / N-plex)

Table 1: System Protocols and Inter-Process Specifications

Protocol	Port	STD	Format	Functional Description
GST IPC	UNIX Pipes	own	Raw	Custom Erlang-to-C99 binary signaling exchange for SDP offers/answers and ICE candidates.
WS Signaling	WSS:8081	own	WSS	Cowboy/N2O room pub/sub channel for user joins, leaves, and chat logs.
QoS Telemetry	WSS:8082	own	WSS	Prioritized client connection statistics pumped to OpenTelemetry collector.
WebRTC SDP	UDP:49232	RFC 8841	SDP	SCTP over DTLS.
WebRTC SDP	UDP:49232	RFC 8866	SDP	Session negotiation declaring codecs (H.264/VP8/Opus) and media directions.
WebRTC ICE	UDP:3478	RFC 8445	STUN/TURN	RTP STUN Binding Attributes Dynamic network connectivity probe establishing optimal media pathways.
TURN Relay	UDP:5349	RFC 8489	STUN/TURN	SRTP media packets through `eturnal` when NAT/firewalls block P2P.
SRTP/SRTCP	UDP:5000	RFC 3711	SRTP/RTCP	Transports secure real-time audio and video tracks between endpoints and GStreamer.

Table 2: JSON Replacement Using ITU-T X-Series Standards

Port	Current Format	Applicable ITU-T X-Recommendation(s)
8081	JSON (N2O)	X.601 (Multi-peer framework), X.602 (Group management), X.609.x (Managed P2P)
Unix	Ad-hoc JSON	X.603 / X.603.1 / X.603.2 (Relayed multicast), X.609.3 / X.609.4 (streaming signalling/peer)
8082	JSON/OTLP	X.609.8 / X.609.10 (Live data sources & data streaming signalling)
Room	JSON events	X.601 / X.602 (Multi-peer & group management)

- **X.604 / X.604.1 / X.604.2** — Mobile multicast communications
- **X.605 – X.608.1** — Enhanced Communications Transport Service (ECTS) for multicast
- **X.609 – X.609.10** — Managed peer-to-peer (P2P) communications (especially X.609.3 / X.609.4 for multimedia streaming signalling and peer protocols)

Implementation Approach in `zencrypted/rtp`

1. Define signaling and control PDUs using ASN.1 (X.680) modules that align with the semantics of X.601 / X.609.x.
2. Use N2O's ASN.1 formatter to replace JSON on WebSocket connections (Port 8081).

3. Extend the C99 GStreamer port (`gst.c`) to use binary messages (inspired by X.603 relayed multicast and X.609.4 peer protocols) over stdin/stdout.

4. For telemetry and room events, adopt structures from X.609.8 / X.609.10.

5. Retain RTP/SRTP for the media plane (already ITU-aligned via RFC 3550).

Expected outcomes: significantly reduced signaling overhead, formal compliance with ITU-T multi-peer frameworks, and improved scalability for high-density rooms.

2.3.3 Detailed Protocol Descriptions

1. WebRTC SDP & ICE (RTP/RTCP Data) The Session Description Protocol (SDP) is exchanged in plain text (RFC 8866) over the WebSocket signaling channel. It acts as the negotiation mechanism for media capabilities, where the GStreamer OS process outputs SDP offers containing its supported codecs (e.g., H.264, VP8, Opus) and payload types. The client browser responds with an SDP answer. Simultaneously, Interactive Connectivity Establishment (ICE) candidates are exchanged to discover the optimal network path. Once the ICE handshake completes, the actual media data is transmitted as encrypted SRTP (Secure Real-time Transport Protocol) and SRTCP packets over UDP.

2. TURN (Traversal Using Relays around NAT) When direct Peer-to-Peer (P2P) UDP connections are blocked by restrictive enterprise firewalls or symmetric NATs, the system falls back to the `eturnal` TURN server. The TURN protocol (RFC 8656) encapsulates the SRTP media packets within its own framed binary format and relays them through port 3478 (UDP or TCP). This guarantees 100% connectivity for all participants, albeit with a slight latency and bandwidth penalty due to the relay overhead.

3. WebSocket API The WebSocket (WS) API serves as the primary signaling and control plane, powered by the Cowboy web server and the N2O framework. It operates over WSS (Port 8081) using a binary-encoded or JSON text payload. The WS connection is stateful and sticky-routed to the specific Erlang node hosting the room. It handles pub/sub events such as `peer_joined`, `peer_left`, chat messages, and the critical SDP/ICE signaling payloads.

4. Telemetry (OTLP) The system employs the OpenTelemetry Protocol (OTLP) to export QoS metrics, traces, and logs. Client browsers and the backend Erlang nodes stream telemetry data (such as packet loss, jitter, and CPU utilization) to the `rtp-telemetry` OTLP Receiver via HTTPS/WSS (Port 8082). The payloads are serialized as Protocol Buffers (Protobuf) or JSON. This data is then ingested by the `otel-collector` and routed to Prometheus and OpenSearch for real-time observability in Grafana.

2.3.4 Signaling Sequence Flow

Figure 2 illustrates the signaling loop during room startup, WebRTC negotiation, media composition, and session finalization.

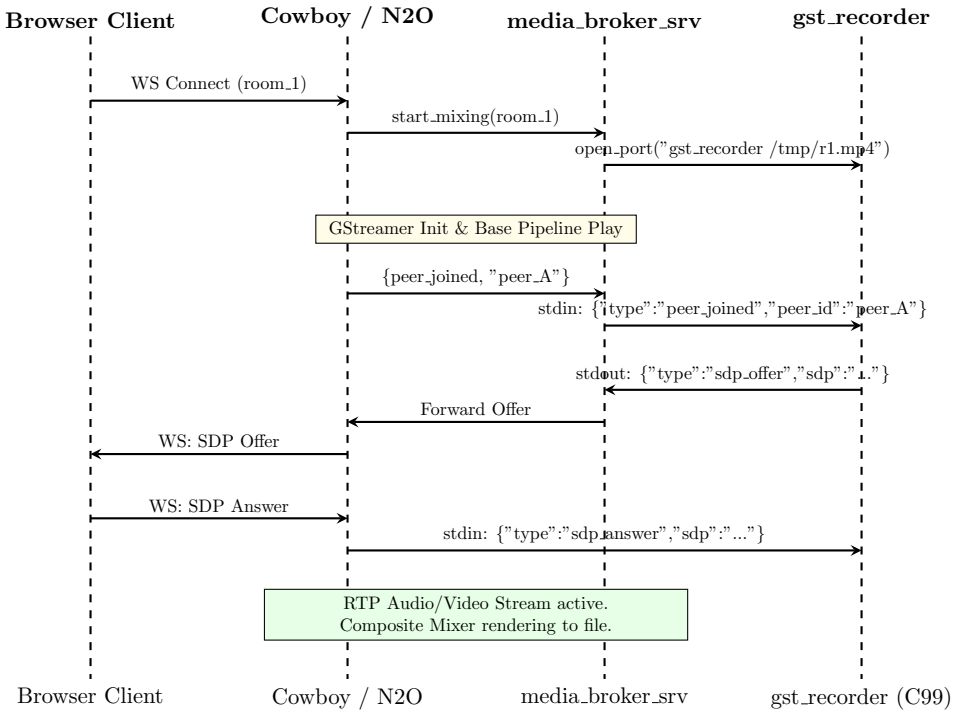

Figure 2: Signaling sequence showing local Port I/O communication between Erlang and the C99 GStreamer binary.

2.4 Multipoint Media Composition Model

The GStreamer media broker executes video grid compositing and audio conferencing concurrently.

2.4.1 Video Matrix Grid Compositing

Incoming H.264/VP8 video packets are decoded into raw YUV frames. The compositor element (`mix`) maps each input stream index $i \in \{0, 1, 2, 3\}$ to a quadrant layout inside a unified 1080p canvas ($W_{\text{out}} = 1920$, $H_{\text{out}} = 1080$). The dimensions of each video frame are scaled to $w = 960$ and $h = 540$. The placement coordinates (x_i, y_i) are formulated as:

$$x_i = (i \pmod 2) \cdot w \tag{2}$$
$$y_i = \lfloor i/2 \rfloor \cdot h \tag{3}$$

The resulting video layout is composed as shown in Equation 4:

$$\text{Canvas}[x, y] = \begin{cases} \text{Stream}_0[x, y] & 0 \le x < 960,\ 0 \le y < 540 \\ \text{Stream}_1[x - 960, y] & 960 \le x < 1920,\ 0 \le y < 540 \\ \text{Stream}_2[x, y - 540] & 0 \le x < 960,\ 540 \le y < 1080 \\ \text{Stream}_3[x - 960, y - 540] & 960 \le x < 1920,\ 540 \le y < 1080 \end{cases} \tag{4}$$

2.4.2 Audio Mixing & Summation

Audio tracks are mixed using GStreamer's `audiomixer` element. Incoming Opus packets are decoded into linear pulse-code modulated (LPCM) audio buffers. The mixed audio signal $S_{\text{mixed}}(t)$ at timestamp t is the sum of the K active participant signals:

$$S_{\text{mixed}}(t) = \sum_{i=1}^{K} S_i(t) \tag{5}$$

To prevent digital clipping/overflow, the mixed LPCM buffer passes through a limiter/clipper stage before being re-encoded into Opus for WebRTC downstream broadcast:

$$S_{\text{output}}(t) = \text{Clamp}(S_{\text{mixed}}(t), -S_{\max}, S_{\max}) \tag{6}$$

3 Capacity Calculation, Telemetry & SRE Metrics

Table 3 specifies the streaming codec, resolution, and bitrates assumed in our capacity review.

Table 3: Codec, Resolution, and Bitrate Parameters

Stream Type	Codec	Resolution	Bitrate (per stream)
Incoming WebRTC (per client)	H.264 (Baseline Profile) or VP8	720p (1280 × 720) @ 30 fps	1.5 Mbps
Output Recording (Composited)	H.264 (High Profile) via x264	1080p (1920 × 1080) @ 30 fps	4.0 Mbps

Under a standard Kubernetes pod boundary of **4 CPU Cores** and **4 GB RAM**, software-based x264 compositing is compute-bound, limiting pod capacity to **3 active mixing rooms** (each room mixer requiring ≈ 1.0 Core). To scale to 50 active rooms, the cluster leverages Ingress-based sharding across **17 stateless replicas**. Offloading the x264 encoder to hardware-accelerated NVENC interfaces on GPU-enabled nodes (reducing per-room complexity to 0.40 Cores) allows a single 20-Core pod to host all 50 rooms.

3.1 System Model & Parameter Definitions

We model the monolithic node as a closed queuing system handling concurrent signaling channels, relayed network packets, and active video compositing pipelines.

3.1.1 Memory Allocation Model

The total memory consumption M_{total} of the container is formulated as:

$$M_{\text{total}} = M_{\text{base}} + N \cdot M_{\text{conn}} + N_{\text{turn}} \cdot M_{\text{turn}} + R \cdot (M_{\text{room}} + M_{\text{gst}}) \quad (7)$$

Where:

- M_{base}: Base RAM overhead of the BEAM runtime and OS libraries ($\approx$ 50 MB).
- N: Total concurrent client connection sessions.
- M_{conn}: Actor memory footprint per socket (server process + state) ($\approx$ 2 KB).
- N_{turn}: Active relayed allocations managed by `eturnal` ($N_{\text{turn}} \leq N$).
- M_{turn}: Network allocation state allocation inside `eturnal` ($\approx$ 10 KB).
- R: Number of active video conference rooms.
- M_{room}: Room coordinator GenServer process memory ($\approx$ 5 KB).
- M_{gst}: RAM footprint of the GStreamer `compositor` OS process ($\approx$ 250 MB).

3.1.2 CPU Utilization Model

The total CPU load C_{total} (expressed in Cores) is defined as:

$$C_{\text{total}} = C_{\text{beam}}(N) + C_{\text{turn}}(T) + R \cdot C_{\text{gst}}(K) \tag{8}$$

Where:

- $C_{\text{beam}}(N)$: CPU load of the BEAM signaling scheduler for N users (highly linear: 1.5×10^{-5} cores per active connection).
- $C_{\text{turn}}(T)$: CPU consumption of STUN/TURN packet forwarding, where T is the throughput in Mbps (Network I/O bound).
- $C_{\text{gst}}(K)$: CPU complexity of a GStreamer composition pipeline with K incoming streams.

3.2 Computational Complexity of GStreamer Compositing

The computational load of GStreamer compositing is a function of incoming resolutions, frame rates, color-space conversions (YUV to RGB and vice-versa), and final compression encoding:

$$C_{\text{gst}}(K) = \sum_{i=1}^{K} D_i(W_i, H_i, F_i) + \Phi\left(\sum_{i=1}^{K} W_i \cdot H_i\right) + E(W_{\text{out}}, H_{\text{out}}, F_{\text{out}}) \tag{9}$$

Where:

- D_i: Decoding complexity of stream i with width W_i, height H_i, and framerate F_i.
- Φ: Grid scaling and pixel blending transformation complexity.
- E: Encoder complexity (e.g., x264 software encoding) for output width W_{out} and height H_{out}.

3.2.1 Media Workload Parameters

To model the CPU and network footprint, the standard streaming codecs, resolutions, and target bitrates for both incoming client WebRTC streams and the output composited recording stream are defined in Table 4.

For a standard layout ($K = 4$ incoming streams at 720p @ 30fps composited into a single 1080p @ 30fps H.264 stream), the CPU core consumption without hardware acceleration (GPU/QuickSync) is calculated empirically as:

$$C_{\text{gst}}(4) \approx 0.15(\text{Dec}) + 0.20(\text{Comp}) + 0.65(\text{x264}) = 1.00\text{Core} \tag{10}$$

Table 4: Codec, Resolution, and Bitrate Parameters

Stream Type	Codec	Resolution	Bitrate (per stream)
Incoming WebRTC (per client)	H.264 (Baseline Profile) or VP8	720p (1280×720) @ 30 fps	1.5 Mbps
Output Recording (Composited)	H.264 (High Profile) via x264	1080p (1920×1080) @ 30 fps	4.0 Mbps

3.3 Capacity Boundaries (4 Cores, 4 GB RAM)

Using the parameters defined above, we evaluate the maximum capacity limits of a single pod.

3.3.1 Scenario A: Idle Signaling & TURN Relay

No GSreamer mixers. Given: $R = 0$, $N = 10,000$, $N_{\text{turn}} = 2,000$ (relaying 2 Gbps aggregate traffic).

$$M_{\text{total}} = 50 \text{ MB} + (10,000 \cdot 2 \text{ KB}) + (2,000 \cdot 10 \text{ KB}) = 90\text{MB} \quad (11)$$

$$C_{\text{total}} = 0.15 \text{ Cores (Erlang)} + 0.80 \text{ Cores (TURN Relay)} = 0.95\text{Cores} \quad (12)$$

Evaluation: The node is highly stable. RAM utilization is at 2.25% and CPU is at 23.7%.

3.3.2 Scenario B: Active Recording & Compositing

Given: $N = 1,000$, $N_{\text{turn}} = 100$, and R active mixing rooms (4 inputs each).

$$M_{\text{total}} = 50 \text{ MB} + 2 \text{ MB} + 1 \text{ MB} + R \cdot 250 \text{ MB} \leq 4,000 \text{ MB} \implies R \leq 15.78 \text{ Rooms}$$
$$C_{\text{total}} = 0.015 \text{ Cores} + 0.04 \text{ Cores} + R \cdot 1.00 \text{ Cores} \leq 4.00 \text{ Cores} \implies R \leq 3.94 \text{ Rooms}$$

Under standard CPU limits, the pod saturates at $R = 3$ active mixed rooms.

3.3.3 Target Scenario: Scaling to 50 Concurrent Rooms

To achieve the operational target of 50 concurrent active rooms, three architectural models are formulated:

1. **Vertical Monolith Node (Scale-Up)**: Host all 50 rooms inside a single monolithic VM/Pod container. Requires $M_{\text{total}} \approx 12.6$ GB RAM and $C_{\text{total}} \approx 50.1$ Cores. Requires a 52-Core / 16 GB RAM bare-metal server node.

2. **Horizontal Scaling (Scale-Out)**: Distribute 50 rooms across P isolated, independent Alpine Linux pods running on a 4-Core / 4 GB RAM boundary. Since $R_{\text{node}} = 3$, $P = \lceil 50/3 \rceil = 17$ Pod Replicas. Nodes operate as share-nothing instances without forming an Erlang cluster.

3. **Hardware-Accelerated Monolith (GPU-Offloaded)**: Offload H.264 software encoding to NVENC. The accelerated CPU complexity per room drops to $C_{\text{gst, gpu}}(4) \approx 0.40$ Cores. Total CPU for 50 rooms is $C_{\text{total}} \approx 20.05$ Cores. Requires a single 20-Core / 16 GB RAM GPU-enabled Kubernetes node with an NVIDIA T4/A10G card.

Extreme Scale Scenario: Scaling to 1000 Concurrent Rooms

At a massive scale of $R = 1,000$ active concurrent rooms (4,000 participants), resource calculations reveal severe hardware limits:

- **Memory Requirement**:

$$M_{\text{b}} = 50\text{MB} \tag{13}$$

$$M_{\text{t}} = (1K \cdot 5\text{KB}) + (1K \cdot 250\text{MB}) \tag{14}$$

$$M_{\text{total}} = M_{\text{b}} + (4K \cdot 2\text{KB}) + (400 \cdot 10\text{KB}) + M_{\text{t}} \approx 250.07\text{GB} \tag{15}$$

- **CPU Core Requirement (x264 Software Encoding)**:

$$C_{\text{total}} = 0.06\text{Cores} + 0.16\text{Cores} + 1,000 \cdot 1.00\text{Core} \approx 1,000.22\text{Cores} \tag{16}$$

- **CPU Core Requirement (GPU-Accelerated NVENC)**:

$$C_{\text{total, gpu}} = 0.06\text{Cores} + 0.16\text{Cores} + 1,000 \cdot 0.40\text{Cores} \approx 400.22\text{Cores} \tag{17}$$

3.4 Systems Soundness & Liveness Analysis

HPA Oscillation (Thundering Herd)

Configuring Kubernetes Horizontal Pod Autoscaling (HPA) against standard CPU limits (e.g., 80% target) creates severe scaling instabilities. When R goes from $1 \to 3$, CPU spikes from $26\% \to 80\%$, triggering replica creation. Because Erlang cluster state is localized on each pod, the new replica registers, but since incoming signaling sockets do not balance instantly, the CPU spike persists on the first node, prompting further HPA scale-ups until replica bounds are hit. The scaling algorithm must utilize custom metrics mapped to active GStreamer mixers (`active_gst_mixers`) instead of generic system CPU metrics.

Local Database Isolation (No Database Clustering)

Under the isolated pod model, Mnesia runs strictly as a local persistent storage database inside each pod's local directory (persisted via Kubernetes PVC on Alpine Linux). Because Erlang nodes do not form a cluster, all database transaction locking, schema updates, and state operations are restricted to the local node. This completely avoids split-brain partitions, node discovery synchronization issues, and cluster-wide database deadlocks during HPA scaling, ensuring high reliability and simplified operations.

Scale-Out Architecture Bottlenecks at 1000+ Rooms

When scaling the cluster horizontally to support $R = 1,000$ active rooms across ≈ 333 replicas of 4-Core pods, three critical infrastructure bottlenecks emerge:

1. **Erlang Distribution Overhead Bypassed**: By routing all room participants to the same pod via Ingress Sticky Session hashing, signaling is fully handled locally in-memory. Consequently, the Erlang distribution mechanism is completely disabled (the nodes run with `-no_distribution`), totally avoiding the quadratic $O(P^2)$ connection storm ($\approx 110,000$ links at $P = 333$ replicas) and ensuring that pods remain as isolated Alpine Linux containers.

2. **Mnesia Transaction Locking Isolated**: Since Mnesia runs as a purely local database on each isolated pod (with local folder persistence on Alpine Linux), transaction lock contention is strictly confined to the local node's processes. This eliminates distributed two-phase commit latency (RTT replication delays) and prevents cluster-wide Mnesia transaction deadlocks.

3. **Network Port & Bandwidth Saturation**: At an average stream bitrate of 1.5 Mbps, the aggregate network ingestion throughput reaches $4,000 \times 1.5$ Mbps $= 6.0$ Gbps. Under standard Kubernetes network overlays (e.g. Calico/Flannel), encapsulation overhead (VXLAN/Geneve) consumes substantial CPU, bottlenecking network card rings. SREs must deploy network interfaces using `hostNetwork` configurations with SRIOV-enabled interfaces to bypass container network encapsulation.

4 Conclusion

The proposed monolithic architecture is highly sound for large-scale signaling and STUN/TURN routing, easily supporting over 10,000 concurrent user events. However, software-based video compositing introduces a severe compute limit, capping density at 3 rooms per pod on a 4-Core boundary. Transitioning to hardware-accelerated transcoding (NVENC/VAAPI) or partitioning media processing to dedicated GPU nodes is required to increase room density.

The `rtp` architecture demonstrates that video conferencing signaling and media composting can be consolidated into a share-nothing Erlang/GStreamer node. By replacing microservice networks and headless browsers with local C99 OS port interfaces and Ingress Room Sticky Sessions, the system avoids cluster synchronization locks and mesh overhead. This provides a highly resilient, cost-effective SRE model for secure networks.